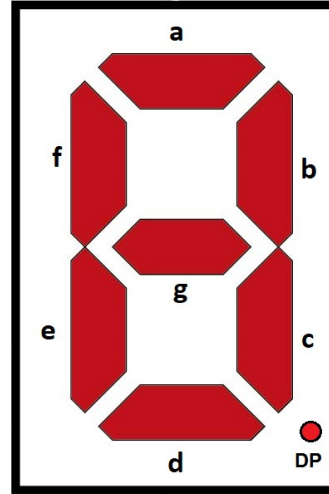
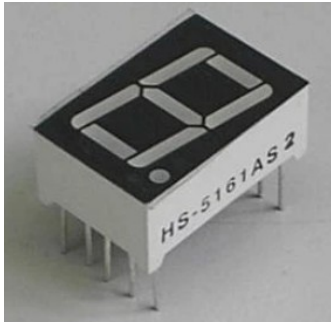


Display de 7 segmentos



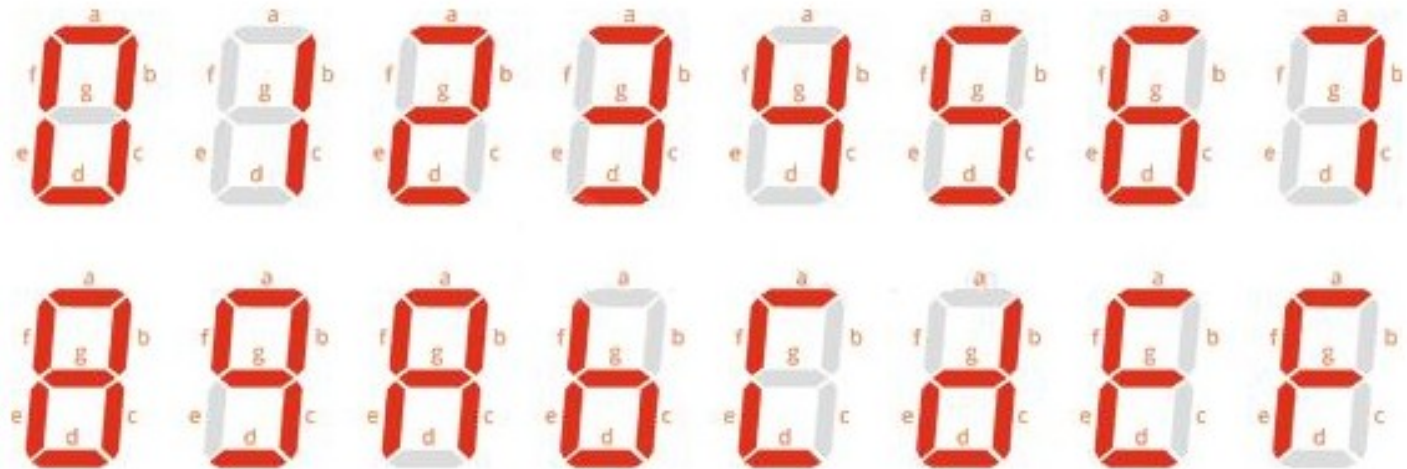
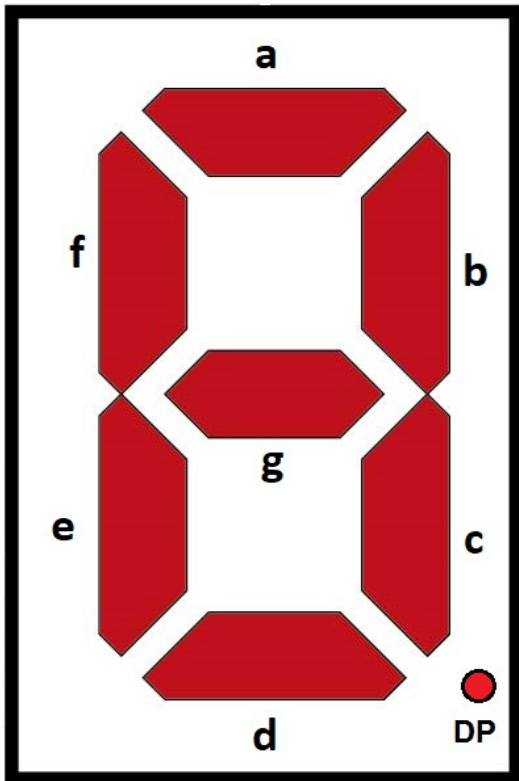
Display de 7 segmentos

- Objetivo: reproduzir algarismos (números) arábicos
 - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9



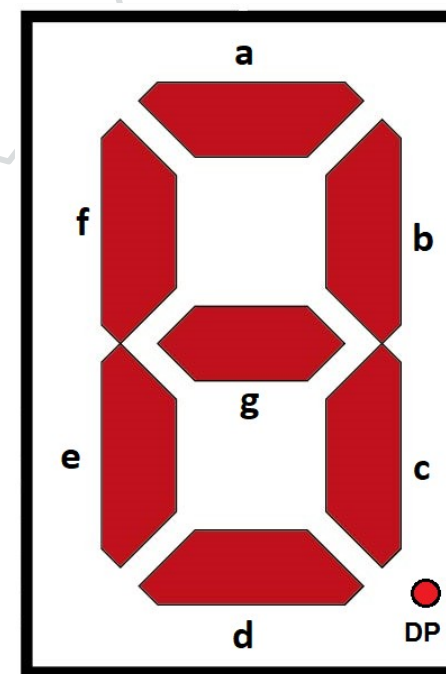
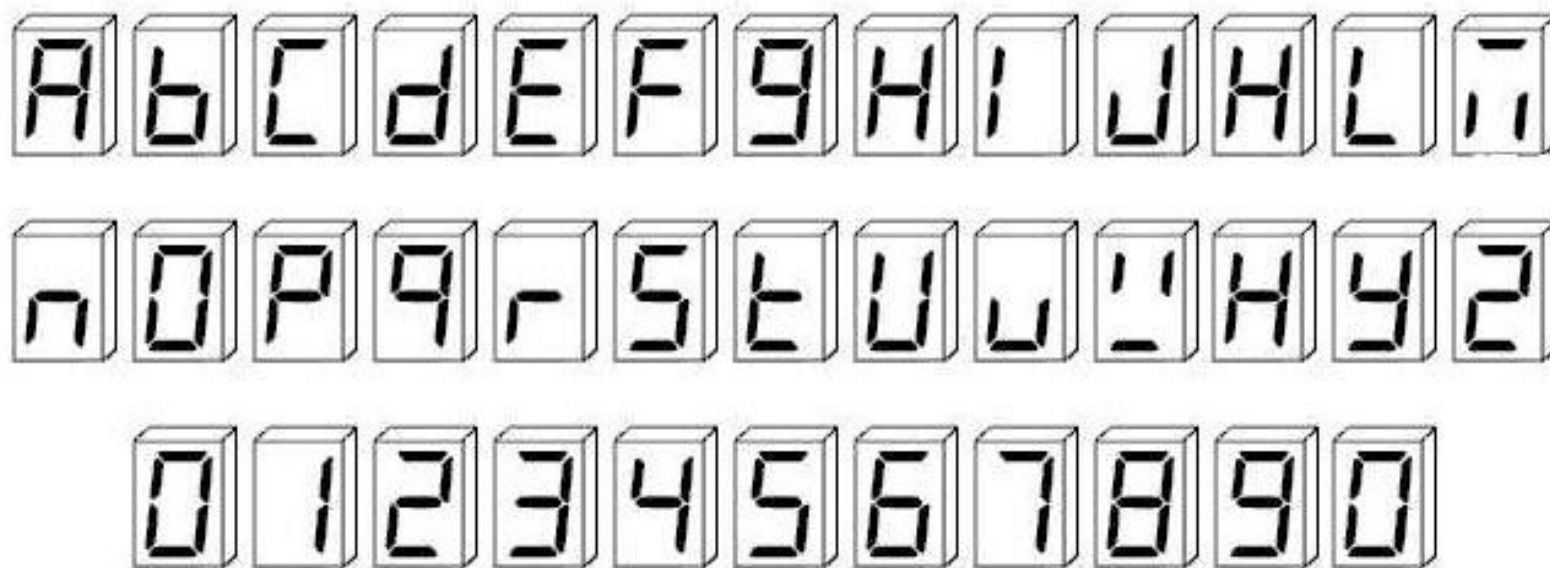
Display de 7 segmentos

- Valores em hexadecimal

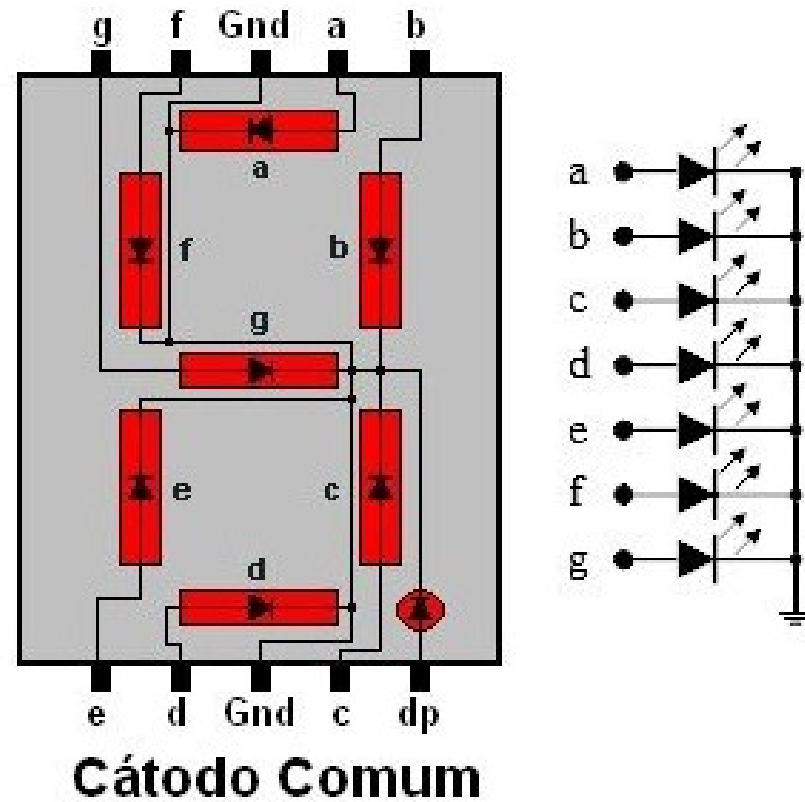
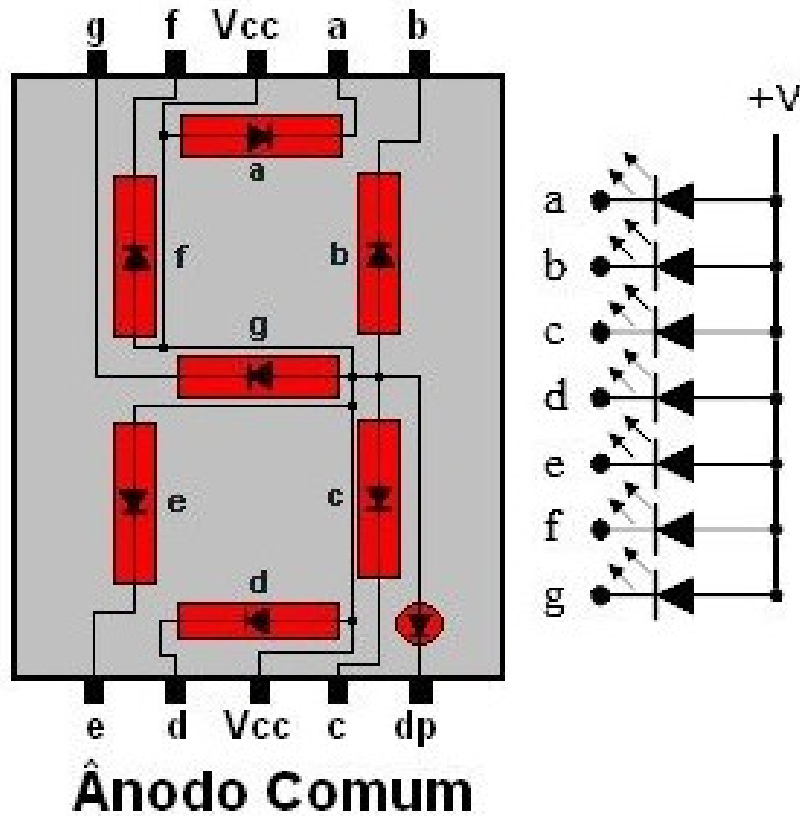


Display de 7 segmentos

- Com muita imaginação é possível escrever um texto

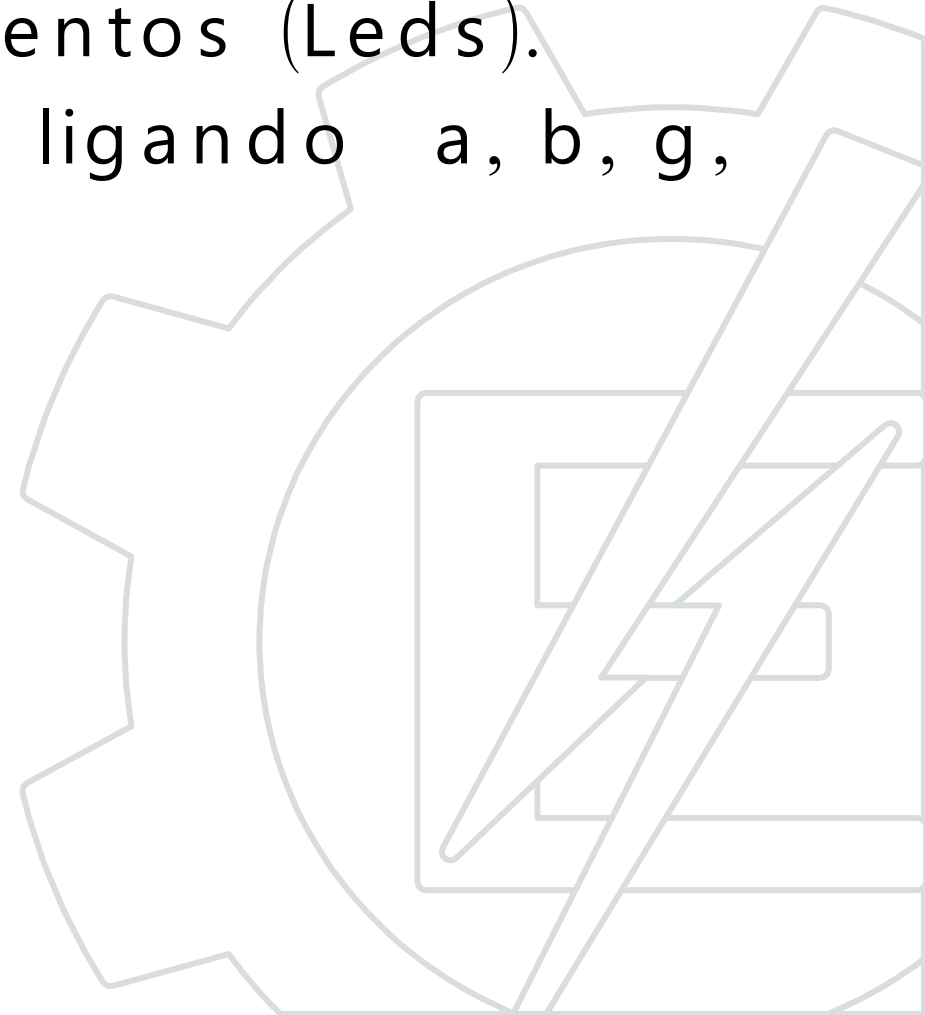
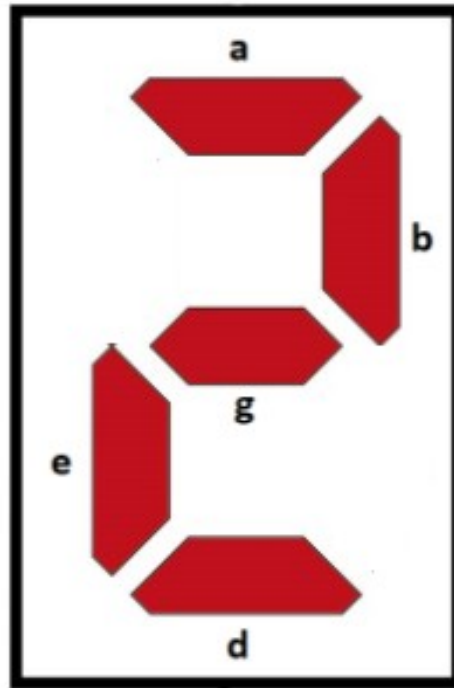


Esquemático de ligação (dois tipos)



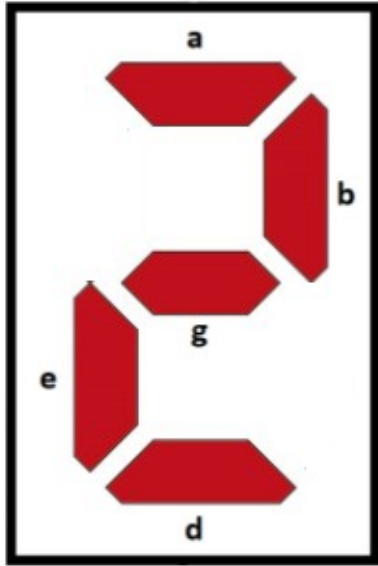
Montagem dos algarismos

- A montagem dos algarismos é feita ligando ou desligando cada um dos 7 segmentos (Leds).
- Exemplo: o número 2 é formado ligando a, b, g, e, d.



Montagem dos algoritmos

Imagine o byte como **gfedcba**, onde cada bit é um segmento.



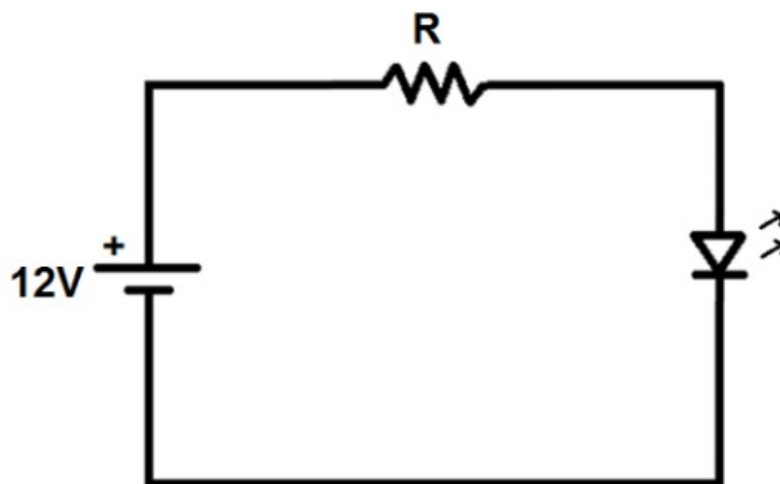
O número 2 é representado por:
0b01011011 ou **0x5B**

Alimentação do display

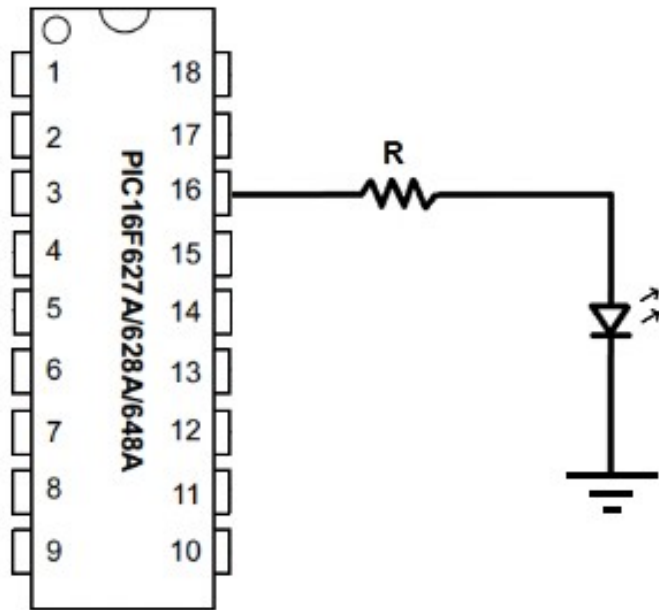


Imagem: www.graphicsfactory.com

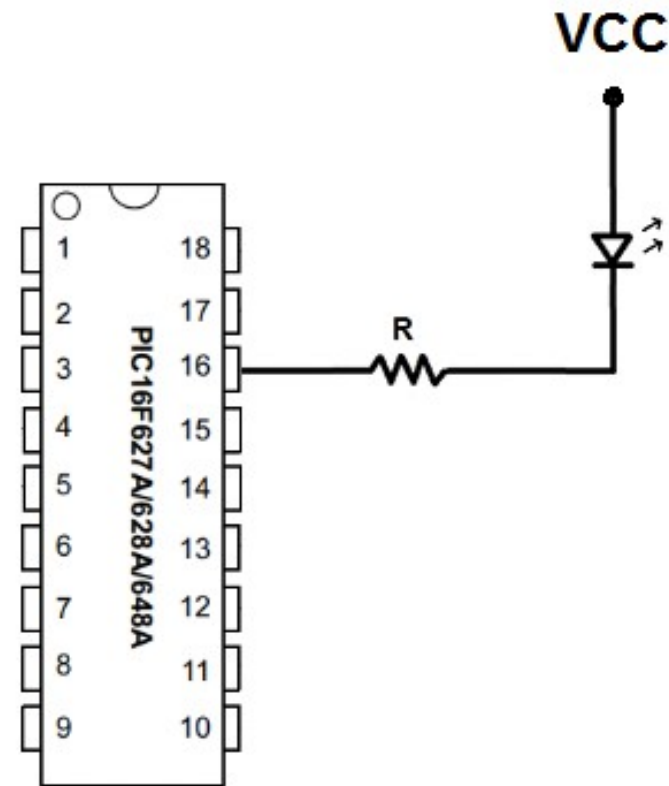
LED não é lâmpada



Alimentação do display



Catodo ao terra



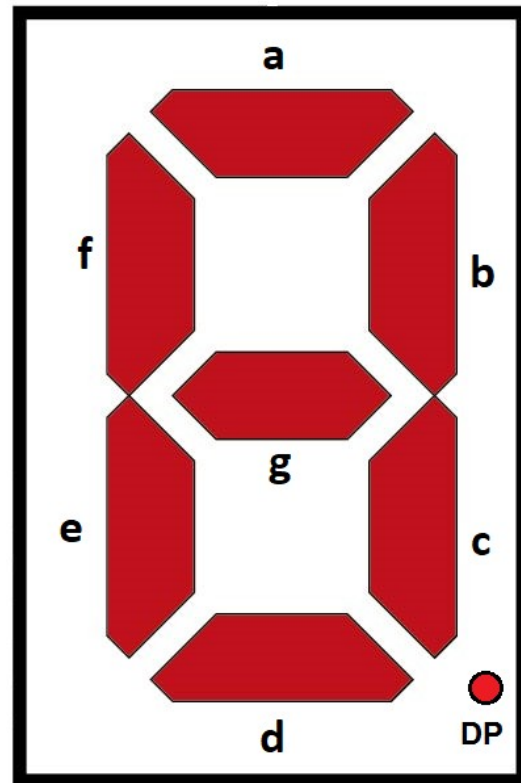
Anodo ao VCC

Formação dos algarismos

Display	g	f	e	d	c	b	a	Hex. (0gfedcba)
0	0	1	1	1	1	1	1	3F
1	0	0	0	0	1	1	0	06
2	1	0	1	1	0	1	1	5B
3	1	0	0	1	1	1	1	4F
4	1	1	0	0	1	1	0	66
5	1	1	0	1	1	0	1	6D
6	1	1	1	1	1	0	1	7D
7	0	0	0	0	1	1	1	07
8	1	1	1	1	1	1	1	7F
9	1	1	0	1	1	1	1	6F
A	1	1	1	0	1	1	1	77
b	1	1	1	1	1	0	0	7C
C	0	1	1	1	0	0	1	39
d	1	0	1	1	1	1	0	5E
E	1	1	1	1	0	0	1	79
F	1	1	1	0	0	0	1	71

Display de 7 segmentos

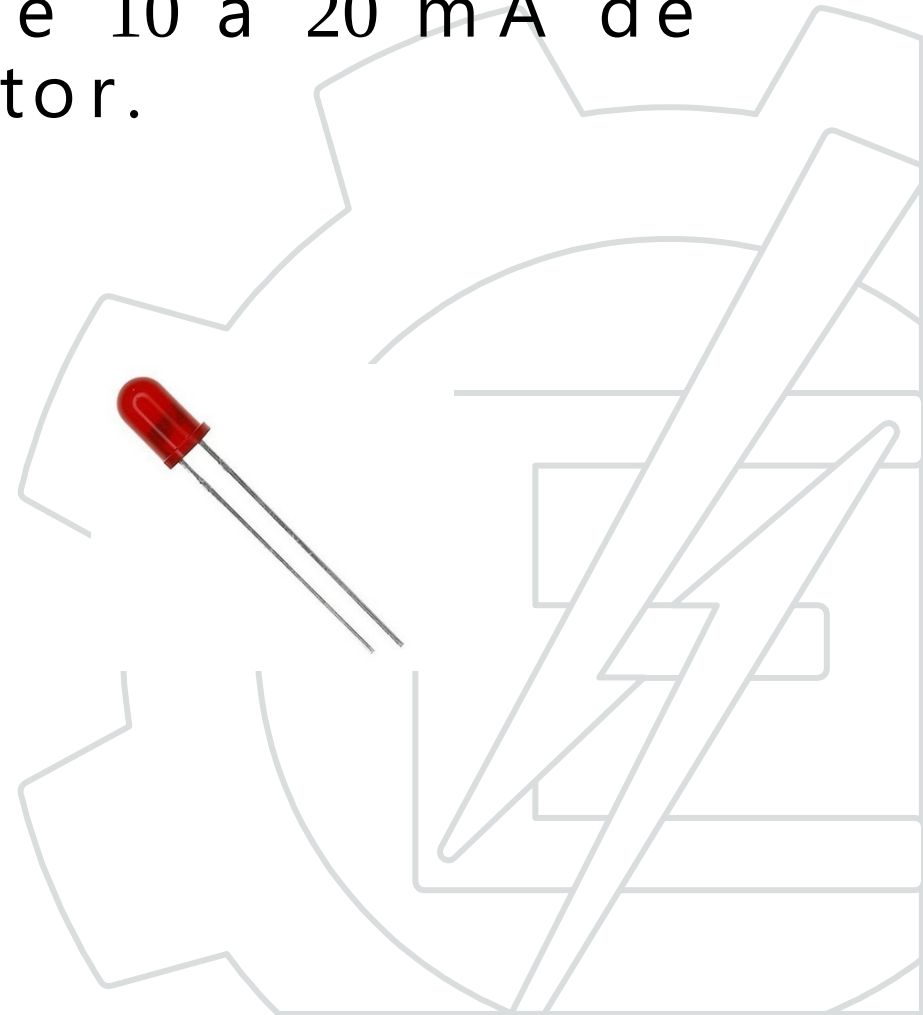
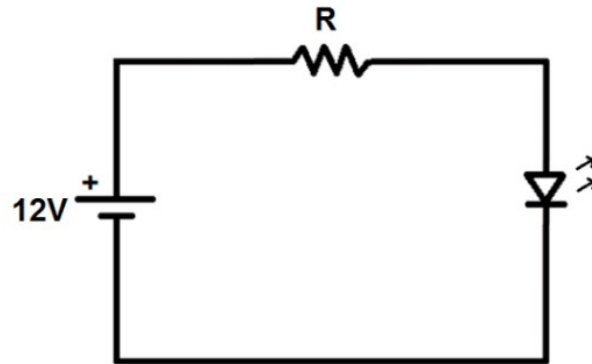
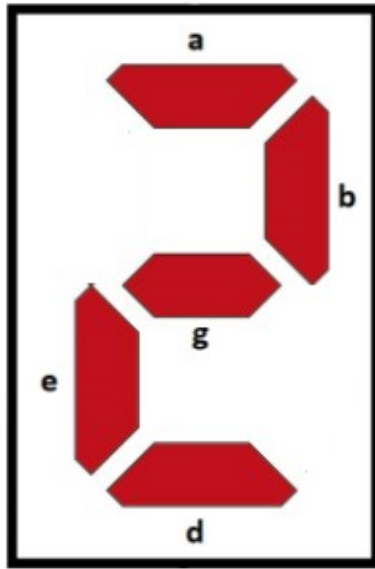
- Alguns displays possuem o ponto decimal embutido. Para ligá-lo basta ligar o bit 7 (MSB).



Ponto decimal

Display de 7 segmentos

- Cada segmento opera como um led portanto cada um consome tipicamente de 10 a 20 mA de corrente e necessita de um resistor.



Exemplo de programa

```
//Includes
```

```
...
```

```
void main(void){
```

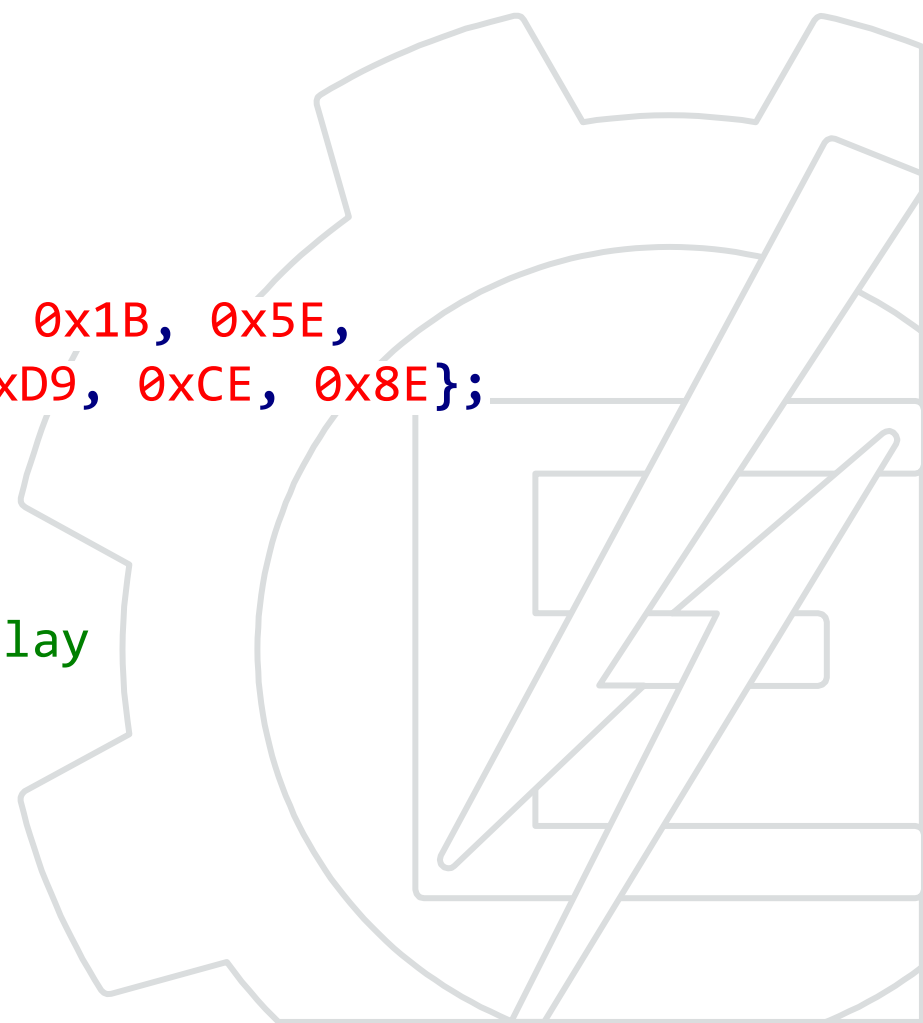
```
    //vetor de conversão
```

```
    const char conv[] = {0xD7, 0x11, 0xCD, 0x5D, 0x1B, 0x5E,  
    0xDE, 0x15, 0xDF, 0x5F, 0x9F, 0xDA, 0xC6, 0xD9, 0xCE, 0x8E};
```

```
    ...
```

```
    PORTD = conv[5]; //Mostra o número 5 no display
```

```
}
```



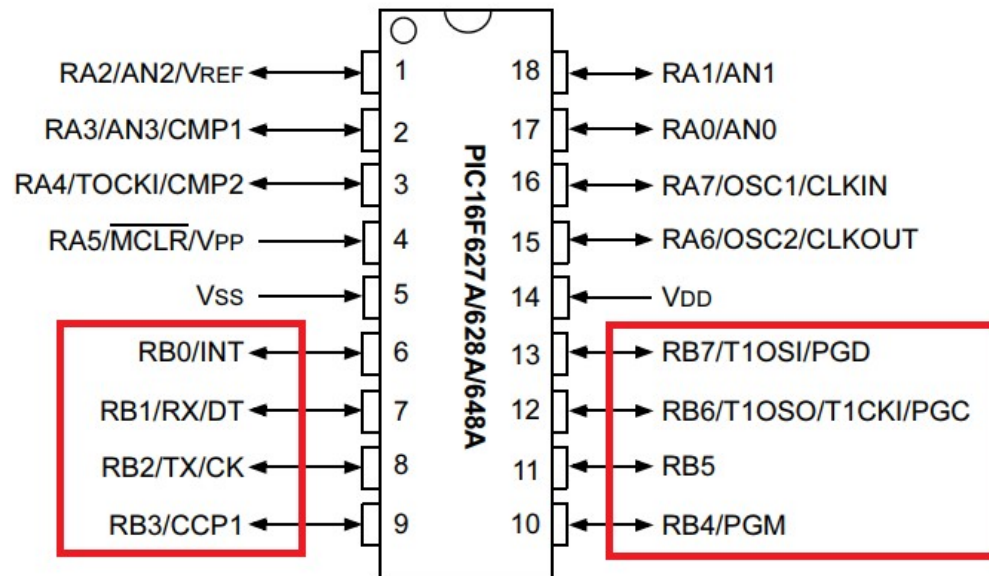
Multiplexação de saídas



Controlando um display

- Considere um display ligado ao PORTB do PIC16F628A

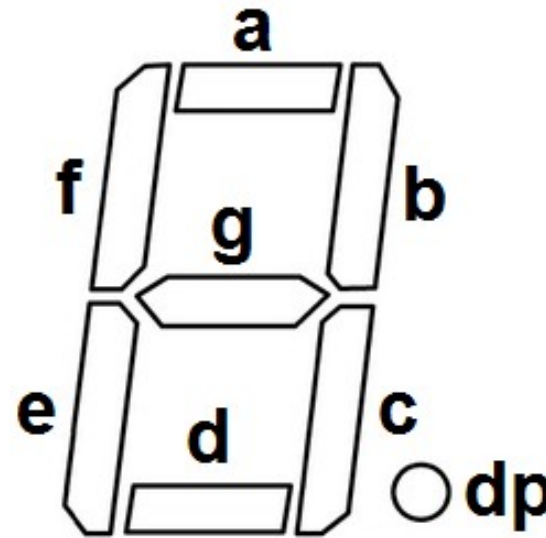
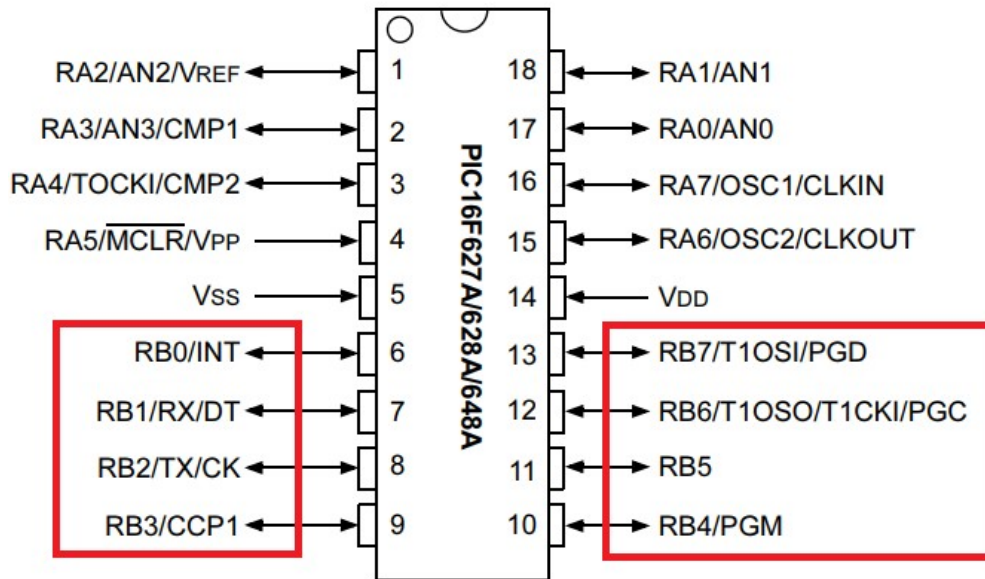
Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR Reset ⁽¹⁾	Details on Page
05h	PORTA	RA7	RA6	RA5	RA4	RA3	RA2	RA1	RA0	xxxx 0000	31
06h	PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0	xxxx xxxx	36
07h	—	Unimplemented								—	—



Controlando um display

- Cada Bit acende um seguimento

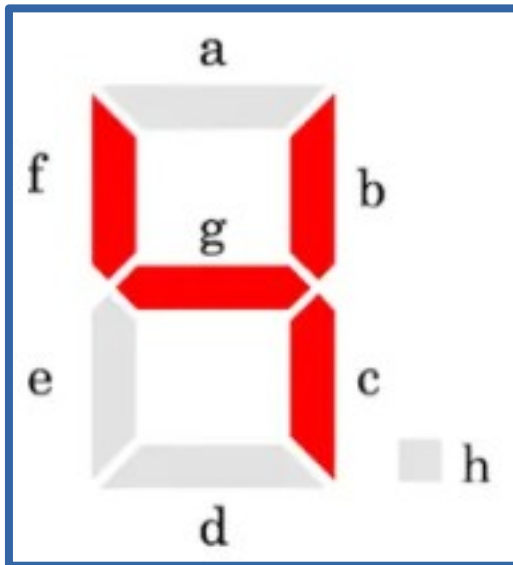
.	g	f	e	d	c	b	a
RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0



Controlando um display

- Cada Bit acende um seguimento

.	g	f	e	d	c	b	a
RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0
0	1	1	0	0	1	1	0



PORTB = 0x66



Controlando um display

- No programa podemos fazer:

```
#define DISPLAY_TRIS  TRISB  
#define DISPLAY      PORTB
```

```
void main(void) {  
    unsigned char x;  
    DISPLAY_TRIS = 0; //define todos os pinos como saída
```

```
    unsigned char numero[] = {  
        0x3F, //0  
        0x06, //1  
        0x5B, //2  
        0x4F, //3  
        0x66, //4  
        0x6D, //5  
        0x7D, //6  
        0x07, //7  
        0x7F, //8  
        0x6F, //9  
    };
```

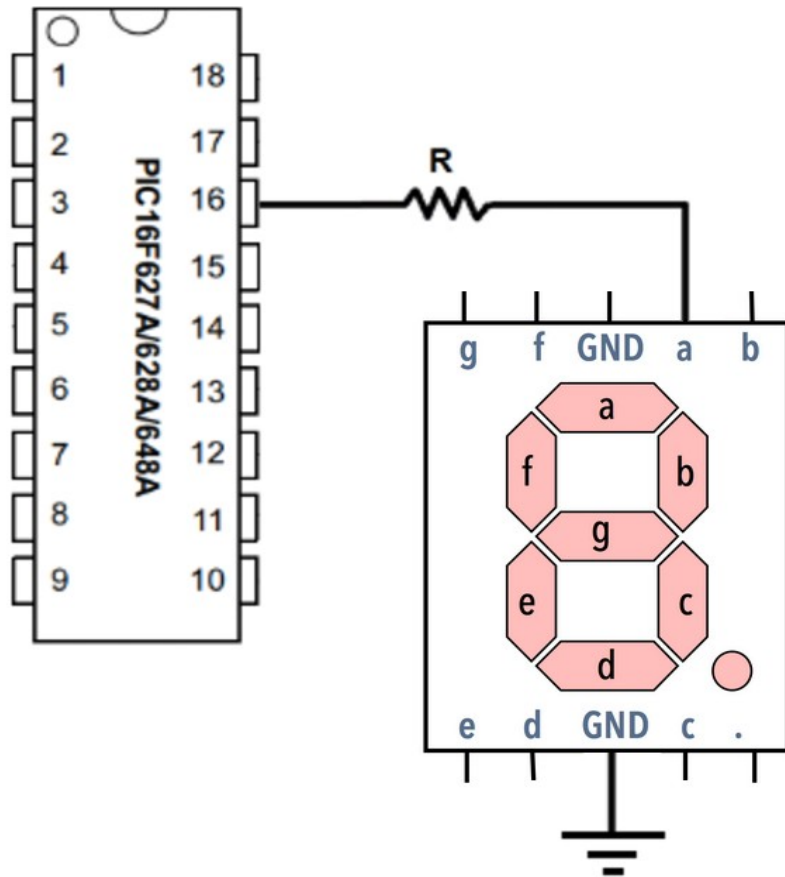
Criando um vetor

```
    while (1)  
    {  
        for (x=0; x<10; x++) {  
            DISPLAY=numero[x];  
            __delay_ms(200);  
        }  
    }  
    return;  
}
```

Escrevendo



Controlando vários displays

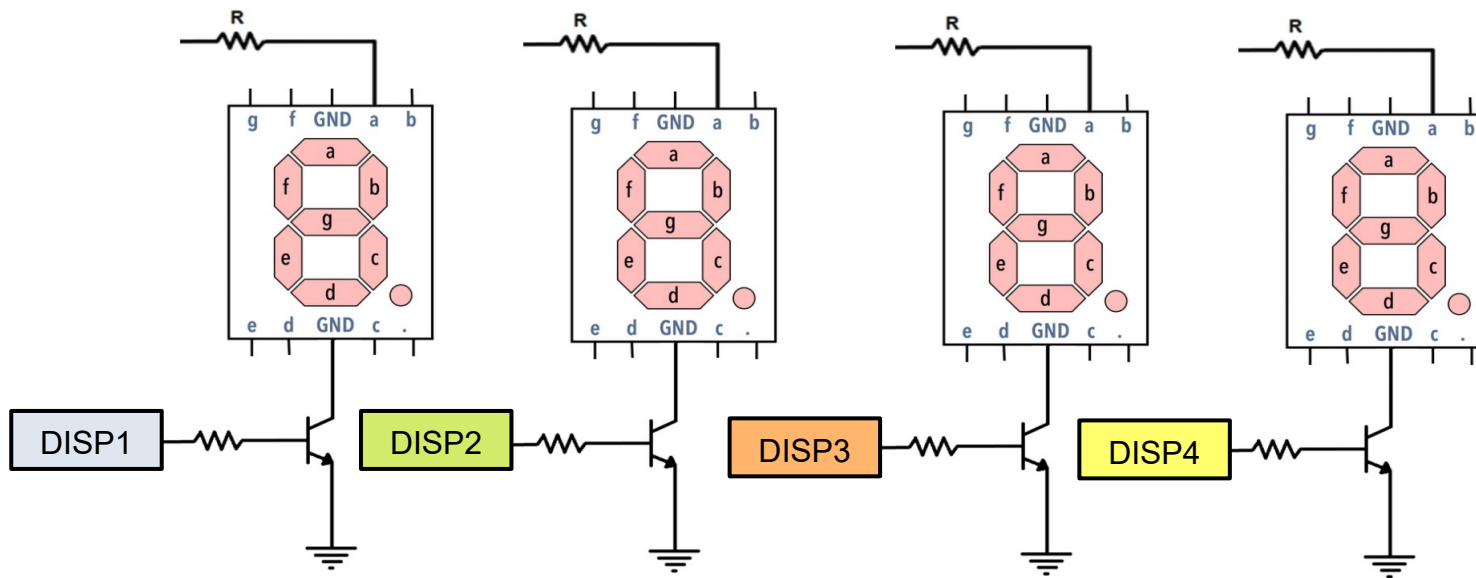


Se cada display
utiliza 7 pinos



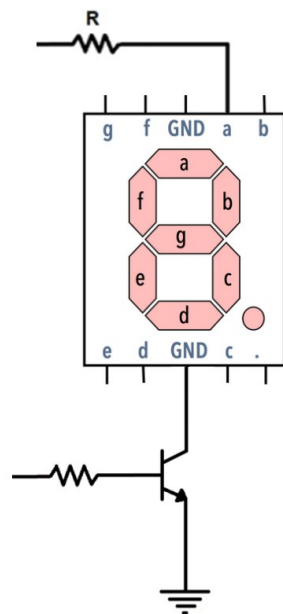
Como controlar vários displays?

Multiplexação de display



Um display é ativado por vez

Multiplexação de display



Se for rápido o olho humano não percebe que apagou



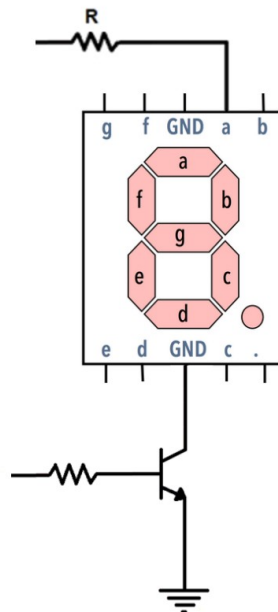
A televisão funciona assim



Multiplexação de display

Atenção

O tempo tem que ser ajustado para evitar cintilação (Flicker)



Multiplexação de display

```
// Valor mostrado
#define DISPLAY_TRIS    TRISB
#define DISPLAY        PORTB
// Posição
#define DIG0_TRIS       TRISAbits.TRISA0
#define DIG0           PORTAbits.RA0
#define DIG1_TRIS       TRISAbits.TRISA1
#define DIG1           PORTAbits.RA1
#define DIG2_TRIS       TRISAbits.TRISA2
#define DIG2           PORTAbits.RA2
#define DIG3_TRIS       TRISAbits.TRISA3
#define DIG3           PORTAbits.RA3
```

```
unsigned char numero[] = {
    0x3F, //0
    0x06, //1
    0x5B, //2
    0x4F, //3
    0x66, //4
    0x6D, //5
    0x7D, //6
    0x07, //7
    0x7F, //8
    0x6F, //9
};
```

Pinos de controle

Vetor de conversão

Multiplexação de display

```
void mostra_valor(unsigned char val, unsigned char posicao)
{
    // Desliga todos os displays
    DIG0 = 0;
    DIG1 = 0;
    DIG2 = 0;
    DIG3 = 0;

    switch (posicao){
        case 0:
            DISPLAY=numero[val];
            DIG0 = 1;
        case 1:
            DISPLAY=numero[val];
            DIG1 = 1;
        case 2:
            DISPLAY=numero[val];
            DIG2 = 1;
        case 3:
            DISPLAY=numero[val];
            DIG3 = 1;
    }
}
```

Recebe valor e posição

Desliga todos

Liga o selecionado

Multiplexação de display

```
while (1)
{
    mostra_valor(1,0);
    __delay_ms(200);
    mostra_valor(2,1);
    __delay_ms(200);
    mostra_valor(3,2);
    __delay_ms(200);
    mostra_valor(4,3);
    __delay_ms(200);
}

return;
```

Liga o primeiro display

Aguarda

Liga o segundo display

...

Multiplexação de display

Onde fica o programa principal?

```
while (1)
{
    mostra_valor(1,0);
    __delay_ms(200);
    mostra_valor(2,1);
    __delay_ms(200);
    mostra_valor(3,2);
    __delay_ms(200);
    mostra_valor(4,3);
    __delay_ms(200);
}

return;
```



Multiplexação de display

```
// Valores atuais do display
unsigned char valor0;
unsigned char valor1;
unsigned char valor2;
unsigned char valor3;
```

Armazenar os valores em
variáveis globais

```
unsigned char display_atual;
```

Guardar qual é o display atual

```
while (1)
{
    mostra_valor();
    __delay_ms(200);
}
```

Substituir o delay
por outras rotinas

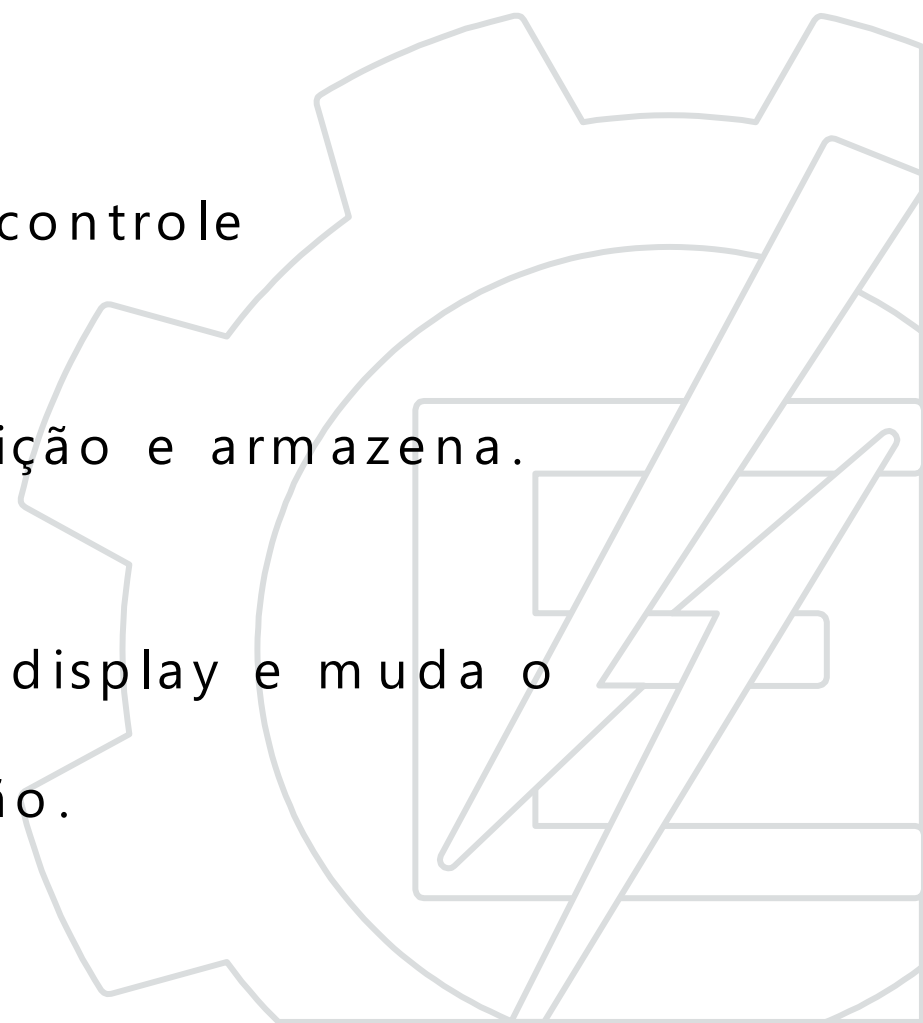
A função mostra_valor() ativa um display por vez

Exemplo de uma biblioteca



Criação da biblioteca

- Funções necessárias para displays multiplexados:
 - `ssdInit()`
 - Configura os terminais de dados e de controle
 - `ssdDigit()`
 - Recebe um número entre 0 e F, a posição e armazena.
 - `ssdUpdate()`
 - Desliga o display atual, liga o próximo display e muda o número.
 - Não pode ter delay dentro dessa função.



```
//includes
```

```
...
```

```
//vetor de conversão
```

```
const char valor[] = {0xD7, 0x11, 0xCD, 0x5D,  
0x1B, 0x5E, 0xDE, 0x15, 0xDF, 0x5F, 0x9F, 0xDA,  
0xC6, 0xD9, 0xCE, 0x8E};
```

```
//armazena qual é o próximo display a ligar  
char display;
```

```
//armazena os valores dos displays  
char v0, v1, v2, v3;
```



```
void ssdDigit(char valor, char posicao){
    if (posicao == 0) v0 = valor;
    if (posicao == 1) v1 = valor;
    if (posicao == 2) v2 = valor;
    if (posicao == 3) v3 = valor;
}
void ssdInit(void){
    v0 = 0; v1 = 0; v2 = 0; v3 = 0;
    //configuracao dos pinos de controle
    BitClr(TRISA, 2);
    BitClr(TRISA, 5);
    BitClr(TRISE, 0);
    BitClr(TRISE, 2);
}
```

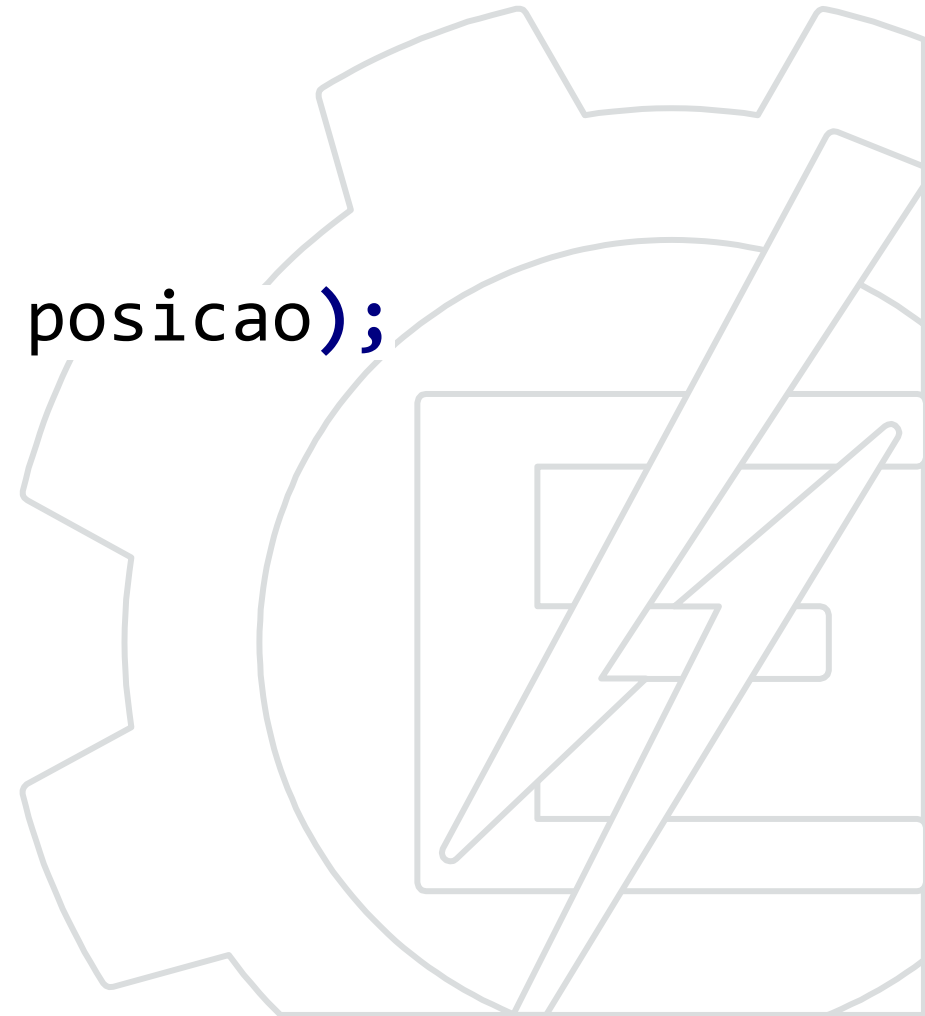


```
void ssdUpdate(void){
    //desliga todos os displays
    PORTA = = 0x00;
    PORTE = = 0x00;
    //desliga todos os segmentos(leds)
    PORTD = = 0x00;
    switch(display){ //liga apenas o display da vez
    case 0: PORTD(valor[v0]); BitSet(PORTA,2);
            display = 1; break;
    case 1: PORTD(valor[v1]); BitSet(PORTA,5);
            display = 2; break;
    case 2: PORTD(valor[v2]); BitSet(PORTE,0);
            display = 3; break;
    case 3: PORTD(valor[v3]); BitSet(PORTE,2);
            display = 0; break;
    default:
            display = 0;
            break;
    }
}
```



Arquivo header (ssd.h)

```
#ifndef DISP7SEG_H
#define DISP7SEG_H
    void ssdDigit(char valor, char posicao);
    void ssdUpdate(void);
    void ssdInit(void);
#endif
```



```
#include "ssd.h"
```

Exemplo de uso

```
void main(void){  
    unsigned int time, count=0, aux=0;  
    ssdInit();  
    for(;;){  
        if(aux==100){ count++; aux=0; }  
        ssdDigit(0, (count    ) %10);  
        ssdDigit(1, (count/10  ) %10);  
        ssdDigit(2, (count/100 ) %10);  
        ssdDigit(3, (count/1000) %10);  
        ssdUpdate();  
        //Ajustar o tempo para evitar o efeito flicker  
        for(time=0;time<100;time++);  
        aux++;  
    }  
}
```



Até a próxima

